

Deliverable 3: Explicit-Feedback ALS Recommender

DS-GA 1004 Big Data Capstone — Goodreads Group cap_44

1 Model objective

We train a latent-factor recommender using Spark’s alternating least squares (ALS) algorithm, using only the explicit Goodreads `rating` field as the preference signal. Rows with `rating = 0` are excluded from training because they encode the absence of an explicit numerical preference rather than a low rating. As required for this deliverable, the implicit fields `is_read` and `is_reviewed` are not used as model inputs.

2 Data, preprocessing, and splits

Model hyperparameters were selected on a deterministic 1% user-based sample of `goodreads_interactions`. After this 1% grid identified `rank=100`, `regParam=0.01` as the best ALS setting, we used a deterministic 3% user-based sample to test whether a larger recommendation candidate pool improved ranking quality for that selected configuration. We also ran a small number of lower-rank 3% sanity checks, but these were not used for model selection. In both samples users were selected by hashing `user_id` and retaining all interactions of sampled users. The raw CSV was converted to Parquet on HDFS as a one-time preprocessing step. Working on a user sample preserves per-user histories, which is required for evaluating ranking recommenders.

Filtering. Users with fewer than 5 interactions in the sample were discarded prior to splitting, since users with very short histories cannot be meaningfully split into observed/held-out evaluation sets.

Split. For each remaining user, interactions were shuffled with a fixed seed (`seed=42`) and partitioned per user into 60% train, 20% validation, and 20% test. This preserves user histories across splits: every evaluation user has both observed interactions (used to fit the model and to filter recommendations) and held-out interactions (used as ground truth). During recommendation, books already present in a user’s training history are removed from that user’s top-100 list so that the model is scored only on its ability to surface unseen items.

3% sample size. The 3% sample contained 6,831,620 interactions from 26,335 users before filtering. After removing users with fewer than 5 interactions, 6,827,622 interactions and 24,415 users remained. The resulting split sizes were 4,086,857 train rows, 1,365,418 validation rows, and 1,375,347 test rows. All three splits retained the same 24,415 users.

3 Training setup

Spark ALS is configured with `userCol=user_id`, `itemCol=book_id`, `ratingCol=rating`, and

`coldStartStrategy="drop"` so that NaN predictions arising from cold users or items at evaluation time are dropped rather than propagated. Training rows are restricted to `rating > 0`.

Each (`rank`, `regParam`) configuration was submitted as a separate Spark job on Dataproc. Each job persisted its validation metrics and the fitted model to HDFS, and a finalizer script aggregated the validation grid into a single Parquet file and selected the best configuration. This isolates jobs from each other and avoids holding many ALS models in memory simultaneously. The 3% artifacts were written to separate HDFS result directories (`explicit_als_sample_3pct_recpool` and `popularity_baseline_sample_3pct`) so that the original 1% outputs remained available for comparison.

4 Evaluation setup

For each evaluable user we generate the top-100 recommendations and score them against held-out interactions using Spark MLlib’s `RankingMetrics`.

Candidate pool. Each evaluable user’s top-100 list is taken from a larger candidate pool generated by `recommendForUserSubset`, with training-history books removed before the top-100 cut. The 1% pipeline uses `recPool=250` throughout; the 3% `recPool` comparison varies this (see Section 8).

Relevance label. A held-out book is treated as relevant for the user if its held-out `rating` ≥ 4 . This matches the explicit-preference semantics of the model and is applied identically when evaluating the popularity baseline used for comparison.

Model-selection metric. We use validation `NDCG@100` as the single criterion for selecting the best ALS configuration. `NDCG` rewards placing relevant items near the top of the ranking and is the most informative ranking metric in this setting.

Reported metrics. We report `MAP`, `NDCG@100`, `Precision@100`, and `Recall@100` as ranking metrics, plus `RMSE` as a secondary rating-prediction metric. Ranking metrics are central, per the deliverable specification.

Additional software. No additional libraries beyond Apache Spark (Spark SQL, Spark MLlib’s `ALS` and `RankingMetrics`) were used for this deliverable.

5 Hyperparameter search

The validation search covered `rank` $\in \{5, 20, 50, 100\}$ and `regParam` $\in \{0.001, 0.01, 0.1, 0.5\}$. We evaluated 13 of the 16 cells (a partial grid focused on the regions that showed clear signal in early runs). The three omitted cells were skipped after neighboring settings made them

Table 1: Validation NDCG@100 across the hyperparameter grid. Best in bold.

rank \ reg	0.001	0.01	0.1	0.5
5	—	0.000172	0.000168	0.000154
20	—	0.002061	0.001441	0.000834
50	0.005863	0.005865	0.002249	0.000809
100	0.014431	0.014463	0.000221	—

unlikely to affect model selection: very low-rank models were already weak, and strong regularization at high rank had already collapsed performance. Table 1 shows validation NDCG@100 for every cell evaluated.

Observations. The grid is wide enough to produce clearly observable differences in the evaluation score. Increasing **rank** from 5 to 100 raises NDCG@100 by roughly two orders of magnitude, indicating that the model lacks sufficient capacity at low rank and benefits substantially from more latent dimensions on this sample. The top two configurations were **rank=100, regParam=0.01** (NDCG@100 = 0.014463) and **rank=100, regParam=0.001** (NDCG@100 = 0.014431). These scores differ by only 0.000032, so they are effectively tied; we selected **regParam=0.01** because it had the highest validation score and provided slightly stronger regularization. Performance collapses at **regParam=0.1**, suggesting that overly strong regularization erases the latent structure that high rank otherwise recovers.

Selected configuration. **rank=100, regParam=0.01.**

6 Validation and test performance

Final metrics for the selected model are shown in Table 2. Validation NDCG@100 is the same number bolded in Table 1; test metrics are slightly higher than validation across the board, which is typical for this size of held-out partition and does not suggest obvious overfitting on this split.

Table 2: Selected explicit-ALS model (**rank=100, regParam=0.01**) on validation and test.

Split	MAP	NDCG@100	P@100	R@100	RMSE
Val.	0.002810	0.014463	0.003379	0.034114	1.9323
Test	0.003041	0.015756	0.003624	0.036810	1.9279

7 Comparison to the popularity baseline

The popularity baseline (Deliverable 2) is evaluated on the same 1% sample, the same train/val/test split, and the same **rating** ≥ 4 relevance label, making the comparison directly comparable.

The explicit-feedback ALS model underperforms the popularity baseline by roughly a factor of 5–10 across

Table 3: Test-set comparison: popularity baseline vs. explicit-feedback ALS.

Model	MAP	NDCG@100	P@100	R@100
Popularity	0.034330	0.107032	0.020639	0.176481
Expl. ALS	0.003041	0.015756	0.003624	0.036810

every ranking metric on the 1% sample. The most plausible explanation is structural: on a small user sample, the average book has very few explicit ratings, so the ALS factorization has little per-item signal from which to learn book factors, especially for the long tail of less-rated books. The popularity baseline, in contrast, exploits Goodreads’ heavy-tailed read distribution, where a small set of books dominates user histories regardless of sample size; recommending those popular books to every user is therefore a strong default. More explicit-rating signal may narrow this gap, but the 3% rerun below shows that moving the selected ALS configuration to a larger user sample did not improve top-100 ranking performance.

8 Scaling to a 3% sample

Based on the 1% results, we moved the selected ALS configuration to a larger 3% user sample to probe how the model scales and where the practical bottlenecks lie. Since **rank=100, regParam=0.01** had already been selected on the 1% validation grid, the main 3% experiment varied only the recommendation-pool size (**recPool** $\in \{250, 500\}$). Here, **recPool** is the number of candidate items generated per evaluable user from **recommendForUserSubset** before removing training-history books and taking the top 100. **maxIter** was held at 5 throughout to keep wall-clock cost manageable.

Table 4: 3% validation comparison for the selected ALS hyperparameters (**rank=100, regParam=0.01**).

recPool	MAP	NDCG@100	R@100
250	0.001363	0.006217	0.014823
500	0.001368	0.006303	0.014890

Selected 3% evaluation setting. Using validation NDCG@100 as the criterion for the candidate-pool comparison, **recPool=500** was slightly better than **recPool=250**. On the 3% test split, the selected 3% setting (**rank=100, regParam=0.01, recPool=500, maxIter=5**) achieved MAP = 0.00158, NDCG@100 = 0.00678, P@100 = 0.00158, R@100 = 0.01583, and RMSE = 1.679.

Table 5: 3% test-set comparison using the shared **rating** ≥ 4 relevance label.

Model	MAP	NDCG@100	P@100	R@100
Popularity	0.034398	0.107278	0.020638	0.175361
Expl. ALS	0.001581	0.006778	0.001584	0.015833

Observations. Doubling `recPool` from 250 to 500 produced only a marginal validation NDCG improvement (0.00622 vs. 0.00630), suggesting that candidate-pool truncation is not the main bottleneck for this model. We also ran a small number of lower-rank 3% sanity checks at `rank` $\in \{20, 50\}$ and `regParam` $\in \{0.001, 0.01\}$; those runs preserved the 1% conclusion that `rank` was the dominant hyperparameter and that lower-rank models performed substantially worse.

Second, absolute ranking quality on the 3% sample is lower than what was observed on the 1% sample (test NDCG@100 of 0.0068 vs. 0.0158). This is initially counterintuitive (more data can help ALS), but we did not run the controlled experiments needed to isolate the cause. Plausible contributors include: (i) *catalog/held-out scaling*: the 3% sample exposes a larger active item catalog and a longer per-user held-out tail, so a fixed top-100 cut covers a smaller fraction of relevant items and depresses recall-based metrics; and (ii) *sample variance*: a single 1% and a single 3% hash sample do not separate a real scaling trend from differences in the active user/book mix. The popularity baseline is also affected by the sampled user/book mix, so the more meaningful quantity at each scale is the relative gap between ALS and popularity rather than the absolute number; identifying which factor dominates would require additional controlled experiments.

Limitations and future work. Scaling exposed several practical constraints, each suggesting a concrete fix for the full-dataset analysis. (i) `recommendForUserSubset` dominated evaluation cost: generating a 500-item pool per evaluable user forces a large dense factor product that on the 3% sample consumed more shuffle and executor memory than training itself, and effectively capped how wide a grid we could explore. This is why the 3% experiment focused on candidate-pool size for the selected ALS configuration rather than repeating the full 1% hyperparameter grid. Replacing exhaustive candidate generation with approximate maximum-inner-product search over item factors could reduce the cost of producing candidate recommendations at larger catalog sizes. (ii) Per-user held-out sets are sparse, so per-user ranking metrics have high variance, and even a 3% sample yields a noisy estimate of NDCG. Attaching bootstrap confidence intervals to per-user ranking metrics would let the popularity comparison reflect sampling noise. (iii) We were constrained to `maxIter=5` for the same compute reasons, which may understate the asymptotic quality of higher-rank models that benefit from more ALS sweeps. Raising `maxIter` to 10–20 and re-tuning `regParam` at that operating point would test this. (iv) Cold items are silently removed by `coldStartStrategy="drop"` and the dropped fraction was not measured directly, which could affect the ALS-vs-popularity comparison at scale. Explicit drop-rate accounting would calibrate

evaluation between sample sizes.

9 Summary

The explicit-feedback ALS recommender was trained on the rating signal alone, with `rank` and `regParam` tuned on validation NDCG@100. The best configuration (`rank=100`, `regParam=0.01`) achieved validation NDCG@100 = 0.0145 and test NDCG@100 = 0.0158 on the 1% sample, but did not outperform the popularity baseline. Moving this selected configuration to a 3% sample and increasing `recPool` from 250 to 500 produced only a marginal validation improvement, and the 3% test result still underperformed the 3% popularity baseline (test NDCG@100 = 0.1073). This result is consistent with a larger active catalog, sparse explicit ratings, and a tighter compute budget (`recPool=500`, `maxIter=5`), but the exact cause was not isolated. The experiments confirm that `rank` is the dominant model hyperparameter and that candidate-generation cost, not training, is the binding constraint for scaling further.